Project - Build a Student Grade & Honor Roll Tracker

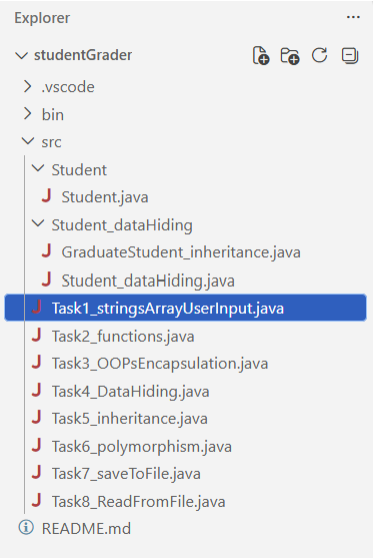
Task 1: Read messy Student Names from user
Task 2: Function Demonstration
Task 3: OOPs concepts – class, encapsulation
Task 4: OOPs concepts – Data hiding
Task 5: OOPs – Inheritance and overriding
Task 6: OOPS – Polymorphism
Task 7: Save to File
Task 8: Read back from on start-up

Project - Build a Student Grade & Honor Roll Tracker

This project bridges your fundamental knowledge (I/O, logic, and functions) with the next set of essential Java concepts: Strings, Arrays, Wrapper Classes/Type Casting, and Basic Classes/Objects, inheritance and Polymorphism.

The Goal

Build an application that reads a student's raw data string from user.
Extracts their numeric grades into an array, calculates their average, and generates an automated report profile card.
This UnifiedTracker.java step-by-step evolution of GradeTracker application.



Task1_stringsArrayUserInput.java

src > Task1_stringsArrayUserInput.java

```java
import java.util.Scanner;

public class Task1_stringsArrayUserInput {

    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Task 1: String Input & Cleaning ===");
        System.out.print("Enter student name (e.g., '  john doe  '): ");
        String rawName = inputScanner.nextLine();

        String cleanName = rawName.trim();
        String formattedName = cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
        System.out.println("Saved Name: '" + formattedName + "'");

        System.out.println("\n=== Task 2: Parsing & Casting ===");
        System.out.print("Enter text grade for Math: ");
        String mathText = inputScanner.next();
        System.out.print("Enter text grade for Science: ");
        String scienceText = inputScanner.next();

        int mathScore = Integer.parseInt(mathText);
        int scienceScore = Integer.parseInt(scienceText);

        double initialAverage = (double) (mathScore + scienceScore) / 2;
        System.out.println("Initial 2-Class Average: " + initialAverage + "%");

        // ADDING TASK 3 CODE DIRECTLY BELOW
        System.out.println("\n=== Task 3: Arrays & Loops ===");
        System.out.print("How many total classes do you want to track? ");
        int totalClasses = inputScanner.nextInt();

        int[] scheduleGrades = new int[totalClasses];
        int arraySum = 0;

        for (int i = 0; i < scheduleGrades.length; i++) {
            System.out.print("Enter text grade for class #" + (i + 1) + ": ");
            String textGrade = inputScanner.next();
            // Reuse Task 2 wrapper parsing directly inside our array tracking loop
            scheduleGrades[i] = Integer.parseInt(textGrade);
            arraySum += scheduleGrades[i];
        }

        double finalAverage = (double) arraySum / scheduleGrades.length;
        System.out.println("Final Schedule Average: " + finalAverage + "%");

        inputScanner.close();
    }
```

Task2_functions.java

src > Task2_functions.java

```java
// Save this entire file as: Task2_functions.java
import java.util.Scanner;

public class Task2_functions {

    // ═══ FUNCTION 1: Now handles the console input, cleans it, and returns a String ═══
    public static String formatName(Scanner inputScanner) {
        System.out.print("Enter student name (e.g., '  john doe  '): ");
        String rawName = inputScanner.nextLine();

        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    // ═══ FUNCTION 2: Now handles the console inputs, parses them, and returns a double ═══
    public static double parseAndCalculateInitial(Scanner inputScanner) {
        System.out.print("Enter text grade for Math: ");
        String textGrade1 = inputScanner.next();
        System.out.print("Enter text grade for Science: ");
        String textGrade2 = inputScanner.next();

        int score1 = Integer.parseInt(textGrade1);
        int score2 = Integer.parseInt(textGrade2);
        return (double) (score1 + score2) / 2;
    }

    // ═══ FUNCTION 3: Takes the array, calculates, and returns a double ═══
    public static double calculateAverage(int[] gradesArray) {
        int sum = 0;
        for (int score : gradesArray) {
            sum += score;
        }
        return (double) sum / gradesArray.length;
    }

    // ═══ FUNCTION 4: Handles getting the size, creates, populates, and returns the array ═══
    public static int[] fillGradesArray(Scanner inputScanner) {
        System.out.print("How many total classes do you want to track? ");
        int totalClasses = inputScanner.nextInt();
        int sum = 0;

        int[] scheduleGrades = new int[totalClasses];
        for (int i = 0; i < scheduleGrades.length; i++) {
            System.out.print("Enter text grade for class #" + (i + 1) + ": ");
            String textGrade = inputScanner.next();
            scheduleGrades[i] = Integer.parseInt(textGrade);
            sum += scheduleGrades[i];
        }
        return scheduleGrades;
    }

    // ═══ CLEAN MAIN EXECUTIVE RUNNER ═══
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        // --- TASK 1 CUMULATIVE REUSE ---
        System.out.println("=== Task 1: String Input & Cleaning ===");
        String formattedName = formatName(inputScanner); // Passed Scanner, returns String
        System.out.println("Saved Name: '" + formattedName + "'");

        // --- TASK 2 CUMULATIVE REUSE ---
        System.out.println("\n=== Task 2: Parsing & Casting ===");
        double initialAverage = parseAndCalculateInitial(inputScanner); // Passed Scanner, returns double
        System.out.println("Initial 2-Class Average: " + initialAverage + "%");

        // --- TASK 3 CUMULATIVE REUSE (With Loop Replaced) ---
        System.out.println("\n=== Task 3 & 4: Array Size & Loop Function ===");
        int[] scheduleGrades = fillGradesArray(inputScanner); // Passed Scanner, returns int[]

        // --- TASK 4 OUTPUT VERIFICATION ---
        System.out.println("\n=== Task 4: Final Calculated Return Values ===");
        double computedAverage = calculateAverage(scheduleGrades); // Computes average
        System.out.println("Clean Function Calculated Average: " + computedAverage + "%");

        inputScanner.close();
    }
}
```

Student.java

src > Student > Student.java

```java
package Student;

import java.util.Scanner;

public class Student {
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    public String name;
    public int[] grades;

    public Student(String rawName, Scanner inputScanner) {
        this.name = formatName(rawName);
        this.grades = new int[SUBJECTS.length];
        fillGradesArray(inputScanner);
    }

    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    private void fillGradesArray(Scanner inputScanner) {
        System.out.println("\n--- Entering Grades for Class Schedule ---");
        for (int i = 0; i < this.grades.length; i++) {
            System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
            String textGrade = inputScanner.next();
            try {
                this.grades[i] = Integer.parseInt(textGrade);
            } catch (NumberFormatException e) {
                System.out.println("Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
                this.grades[i] = 0;
            }
        }
    }

    public double calculateAverage() {
        int sum = 0;
        for (int score : this.grades) {
            sum += score;
        }
        return (double) sum / this.grades.length;
    }
}

```

Student.java | GraduateStudent_inheritance.java

src > Student > GraduateStudent_inheritance.java

```java

package Student_dataHiding;

// ═══ INHERITANCE (CHILD CLASS) ═══
class GraduateStudent extends Student_dataHiding {

    private String thesisTitle;

    // Child Constructor calling parent super
    public GraduateStudent(String rawName, String thesisTitle, Scanner inputScanner) {
        super(rawName, inputScanner);
        this.thesisTitle = thesisTitle;
    }

    public String getThesisTitle() {
        return this.thesisTitle;
    }

    // ═══ NEW STEP: METHOD OVERRIDING ═══
    @Override
    public boolean qualifiesForHonorRoll() {
        // Graduate students must hit a tougher threshold (95.0%)
        // Notice it calls the inherited calculateAverage() method seamlessly!
        return calculateAverage() >= 95.0;
    }
}
```

Student.java | Student_dataHiding.java

src > Student_dataHiding > Student_dataHiding.java

```java

public class Student_dataHiding {
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    private String name;
    private int[] grades;

    public Student_dataHiding(String rawName, Scanner inputScanner) {
        this.name = formatName(rawName);
        this.grades = new int[SUBJECTS.length];
        fillGradesArray(inputScanner);
    }

    public String getName() {
        return this.name;
    }

    public int[] getGrades() {
        return this.grades;
    }

    public boolean qualifiesForHonorRoll() {
        return calculateAverage() >= 90.0;
    }

    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    private void fillGradesArray(Scanner inputScanner) {
        System.out.println("\n--- Entering Grades for Class Schedule ---");
        for (int i = 0; i < this.grades.length; i++) {
            System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
            String textGrade = inputScanner.next();
            try {
                this.grades[i] = Integer.parseInt(textGrade);
            } catch (NumberFormatException e) {
                System.out.println("Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
                this.grades[i] = 0;
            }
        }
    }

    public double calculateAverage() {
        int sum = 0;
        for (int score : this.grades) {
            sum += score;
        }
        return (double) sum / this.grades.length;
    }
}

```

Task3_OOPsEncapsulation.java

src > Task3_OOPsEncapsulation.java

```java


import java.util.Scanner;
import Student.Student;

// MAIN RUNNER EXECUTION SCRIPT
public class Task3_OOPsEncapsulation {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        // --- DATA COLLECTION INTAKE ---
        System.out.println("=== Student Data Entry System ===");
        System.out.print("Enter raw student name: ");
        String rawName = inputScanner.nextLine();

        // --- OBJECT BIRTH SYNTHESIS ---
        // We instantiate the object. It handles text cleaning and sequential
        // subject tracking completely on its own inside the constructor!
        Student activeStudent = new Student(rawName, inputScanner);

        // --- FINAL REPORT CARD PROFILE GENERATION ---
        System.out.println("\n==================================");
        System.out.println("          ACADEMIC REPORT          ");
        System.out.println("==================================");
        System.out.println("Student Profile: " + activeStudent.name);
        System.out.println("Total Classes:    " + activeStudent.grades.length);

        // Call instance method to compute arithmetic output
        double computedAverage = activeStudent.calculateAverage();
        System.out.println("Final Average:    " + computedAverage + "%");
        System.out.println("==================================");

        inputScanner.close();
    }
}
```

Task4_DataHiding.java

src > Task4_DataHiding.java

```java

import java.util.Scanner;
import Student_dataHiding.Student_dataHiding;

// MAIN RUNNER EXECUTION SCRIPT
public class Task4_DataHiding {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        // --- DATA COLLECTION INTAKE ---
        System.out.println("=== Student Data Entry System ===");
        System.out.print("Enter raw student name: ");
        String rawName = inputScanner.nextLine();

        // --- OBJECT BIRTH SYNTHESIS ---
        // We instantiate the object. It handles text cleaning and sequential
        // subject tracking completely on its own inside the constructor!
        Student_dataHiding activeStudent = new Student_dataHiding(rawName, inputScanner);

        // --- FINAL REPORT CARD PROFILE GENERATION ---
        System.out.println("\n==================================");
        System.out.println("          ACADEMIC REPORT          ");
        System.out.println("==================================");
        System.out.println("Student Profile: " + activeStudent.getName());
        System.out.println("Total Classes:    " + activeStudent.getGrades().length);

        // Call instance method to compute arithmetic output
        double computedAverage = activeStudent.calculateAverage();
        System.out.println("Final Average:    " + computedAverage + "%");
        System.out.println("==================================");

        inputScanner.close();
    }
}

```

Task5_inheritance.java

src > Task5_inheritance.java

```java

import java.util.Scanner;
import Student_dataHiding.GraduateStudent_inheritance;

// ═══ MAIN RUNNER EXECUTIVE SCRIPT ═══
public class Task5_inheritance {

    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Graduate Student Data Entry ===");
        System.out.print("Enter raw student name (e.g., '  mariah carey  '): ");
        String rawName = inputScanner.nextLine();

        System.out.print("Enter Research Thesis Topic: ");
        String thesis = inputScanner.nextLine();

        // Child Object Birth
        GraduateStudent_inheritance activeGrad = new GraduateStudent_inheritance(
            rawName, thesis, inputScanner);

        // --- REPORT GENERATION ---
        System.out.println("\n==================================");
        System.out.println("     GRADUATE ACADEMIC REPORT     ");
        System.out.println("==================================");
        System.out.println("Student Profile: " + activeGrad.getName());
        System.out.println("Total Classes:   " + activeGrad.getGrades().length);
        System.out.println("Research Focus:  " + activeGrad.getThesisTitle());

        double avg = activeGrad.calculateAverage();
        System.out.println("Final Average:   " + avg + "%");

        // ═══ EVALUATING THE OVERRIDDEN METHOD ═══
        // Java executes the customized child method logic rules automatically!
        if (activeGrad.qualifiesForHonorRoll()) {
            System.out.println("Honor Roll:      ★ APPROVED (95%+ Target Cleared) ★");
        } else {
            System.out.println("Honor Roll:      Standard Graduate Standing");
        }
        System.out.println("==================================");

        inputScanner.close();
    }
}
```

Task6_polymorphism.java

src > Task6_polymorphism.java

```java
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;
import Student_dataHiding.GraduateStudent_inheritance;
import Student_dataHiding.Student_dataHiding;

// ═══ CONTINUOUS MAIN RUNNER EXECUTION SCRIPT ═══
public class Task6_polymorphism {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("==========================================");
        System.out.println("   WELCOME TO THE GLOBAL STUDENT ENGINE   ");
        System.out.println("==========================================");

        // ═══ DYNAMIC CONTINUOUS WHILE LOOP ═══
        while (true) {
            System.out.println("\nSelect an action:");
            System.out.println("1. Register a Regular Student");
            System.out.println("2. Register a Graduate Student");
            System.out.println("3. Exit Program");
            System.out.print("Enter choice (1-3): ");
            int menuChoice = inputScanner.nextInt();
            inputScanner.nextLine(); // Clear out leftover newline token from scanner buffer

            // ═══ DYNAMIC EXIT JUMP CONDITION ═══
            if (menuChoice == 3) {
                System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
                break; // Instantly breaks the infinite while loop to end the program safely
            }

            // Route user choices or intercept bad choices
            if (menuChoice != 1 && menuChoice != 2) {
                System.out.println("❌ Invalid option. Please select 1, 2, or 3.");
                continue; // Skips the rest of this iteration and restarts the menu loop
            }

            // GATHER SHARED BASE PROFILE DETAILS
            System.out.print("\nEnter raw student name: ");
            String rawName = inputScanner.nextLine();

            // Declare a generic parent type pointer variable to handle polymorphism seamlessly
            Student_dataHiding currentStudent = null;

            // ═══ USER DECIDES THE INSTANTIATION TYPE ═══
            if (menuChoice == 1) {
                // Instantiates a regular student profile
                currentStudent = new Student_dataHiding(rawName, inputScanner);
            } else if (menuChoice == 2) {
                System.out.print("Enter Research Thesis Topic: ");
                String thesis = inputScanner.nextLine();
                // Instantiates a specialized graduate student child profile
                currentStudent = new GraduateStudent_inheritance(rawName, thesis, inputScanner);
            }


            // ═══ POLYMORPHIC REPORT COMPILATION ═══
            System.out.println("\n=====================================================");
            System.out.println("              GENERATING REPORT CARD                 ");
            System.out.println("=====================================================");
            System.out.println("Account File:    " + currentStudent.getName());
            System.out.println("Tracked Classes: " + currentStudent.getGrades().length);

            // If the object is a GraduateStudent, show their thesis topic using a safe type cast check
            if (currentStudent instanceof GraduateStudent_inheritance) {
                GraduateStudent_inheritance castedGrad = (GraduateStudent_inheritance) currentStudent;
                System.out.println("Research Focus:  " + castedGrad.getThesisTitle());
            }

            double averageGrade = currentStudent.calculateAverage();
            System.out.println("Final Grade:     " + averageGrade + "%");

            // Runtime Magic: Automatically triggers the correct 90% or 95% target rules
            if (currentStudent.qualifiesForHonorRoll()) {
                System.out.println("Honor Roll:      ★ APPROVED ★");
            } else {
                System.out.println("Honor Roll:      Standard Academic Standing");
            }
            System.out.println("=====================================================");
        }

        inputScanner.close();
    }
}
```

Task7_saveToFile.java

src > Task7_saveToFile.java

```java
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;
// 🟰 NEW STEP: IMPORT PERMANENT STORAGE FILE TOOLS 🟰
import java.io.FileWriter;
import java.io.IOException;
import Student_dataHiding.GraduateStudent_inheritance;
import Student_dataHiding.Student_dataHiding;


// 🟰 CONTINUOUS MAIN RUNNER EXECUTION SCRIPT 🟰
public class Task7_saveToFile {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("==========================================");
        System.out.println("   WELCOME TO THE GLOBAL STUDENT ENGINE   ");
        System.out.println("==========================================");

        while (true) {
            System.out.println("\nSelect an action:");
            System.out.println("1. Register a Regular Student");
            System.out.println("2. Register a Graduate Student");
            System.out.println("3. Exit Program");
            System.out.print("Enter choice (1-3): ");
            int menuChoice = inputScanner.nextInt();
            inputScanner.nextLine(); // Clear scanner buffer

            if (menuChoice == 3) {
                System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
                break;
            }

            if (menuChoice != 1 && menuChoice != 2) {
                System.out.println("❌ Invalid option. Please select 1, 2, or 3.");
                continue;
            }

            System.out.print("\nEnter raw student name: ");
            String rawName = inputScanner.nextLine();

            Student_dataHiding currentStudent = null;

            if (menuChoice == 1) {
                currentStudent = new Student_dataHiding(rawName, inputScanner);
            } else if (menuChoice == 2) {
                System.out.print("Enter Research Thesis Topic: ");
                String thesis = inputScanner.nextLine();
                currentStudent = new GraduateStudent_inheritance(rawName, thesis, inputScanner);
            }

            // --- POLYMORPHIC REPORT COMPILATION ---
            System.out.println("\n==================================================");
            System.out.println("              GENERATING REPORT CARD              ");
            System.out.println("==================================================");
            System.out.println("Account File:    " + currentStudent.getName());
            System.out.println("Tracked Classes: " + currentStudent.getGrades().length);

            String extraDetails = "";
            if (currentStudent instanceof GraduateStudent_inheritance) {
                GraduateStudent_inheritance castedGrad = (GraduateStudent_inheritance) currentStudent;
                extraDetails = "Research Focus:  " + castedGrad.getThesisTitle() + "\n";
                System.out.print(extraDetails);
            }

            double averageGrade = currentStudent.calculateAverage();
            System.out.println("Final Grade:     " + averageGrade + "%");

            String honorStatus = "Standard Academic Standing";
            if (currentStudent.qualifiesForHonorRoll()) {
                honorStatus = "⭐ APPROVED FOR HONOR ROLL ⭐";
            }
            System.out.println("Honor Roll:      " + honorStatus);
            System.out.println("==================================================");

            // 🟰 NEW STEP: AUTOMATICALLY SAVE DATA TO HARD DRIVE VIA FILE I/O 🟰
            // Setting the second parameter to 'true' enables APPEND mode so old entries stay saved!
            try (FileWriter fileWriter = new FileWriter("report_cards.txt", true)) {

                // Write data patterns onto the disk exactly how we formatted them on screen
                fileWriter.write("=== STUDENT RECORD ===\n");
                fileWriter.write("Name: " + currentStudent.getName() + "\n");
                fileWriter.write("Average: " + averageGrade + "%\n");
                if (!extraDetails.isEmpty()) {
                    fileWriter.write(extraDetails);
                }
                fileWriter.write("Status: " + honorStatus + "\n");
                fileWriter.write("--------------------------------------\n\n");

                System.out.println("💾 Backup Status: Report successfully written to 'report_cards.txt'.");

            } catch (IOException e) {
                // Intercept data path anomalies (like read-only file lockouts or hardware drops)
                System.out.println("❌ Local Storage Failure: Could not write record backup to disk drive.");
            }
        }

        inputScanner.close();
    }
}
```

Task8_ReadFromFile.java

src > Task8_ReadFromFile.java

```java

import java.util.Scanner;
import Student_dataHiding.GraduateStudent_inheritance;
import Student_dataHiding.Student_dataHiding;
import java.io.FileWriter;
import java.io.IOException;
// NEW STEP: IMPORT THE FILE READ UTILITY
import java.io.File;

// CONTINUOUS MAIN RUNNER EXECUTION SCRIPT
public class Task8_ReadFromFile {

    // NEW STEP: FUNCTION TO LOAD PAST RECORDS FROM LOCAL DISK
    public static void loadPastRecords() {
        File databaseFile = new File("report_cards.txt");

        // 1. Check if the backup file actually exists on the computer
        if (!databaseFile.exists()) {
            System.out.println("ℹ️ No historical database found. Starting with a blank registry.\n");
            return; // Exit function early if there is nothing to read
        }

        System.out.println("📂 Loading historical archives from disk drive...");
        System.out.println("==================================================");

        // 2. Open a specialized scanner locked onto the physical file instead of System.in
        try (Scanner fileScanner = new Scanner(databaseFile)) {

            // 3. Keep loop active as long as there are text strings left to read in the file
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                System.out.println(line); // Output historical file lines directly to console
            }
            System.out.println("==================================================");
            System.out.println("✅ Data successfully loaded into memory view!\n");

        } catch (IOException e) {
            // Guard against system errors during active read runtime
            System.out.println("❌ Local Storage Failure: Could not safely parse text files.");
        }
    }
```

```
    public static void main(String[] args) {
        // ┌─┐ BOOT-UP REUSE: FETCH OLD HISTORY THE MOMENT APPLICATION STARTS └─┘
        loadPastRecords();

        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=========================================");
        System.out.println("   WELCOME TO THE GLOBAL STUDENT ENGINE   ");
        System.out.println("=========================================");

        while (true) {
            System.out.println("\nSelect an action:");
            System.out.println("1. Register a Regular Student");
            System.out.println("2. Register a Graduate Student");
            System.out.println("3. Exit Program");
            System.out.print("Enter choice (1-3): ");
            int menuChoice = inputScanner.nextInt();
            inputScanner.nextLine(); // Clear scanner buffer

            if (menuChoice == 3) {
                System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
                break;
            }

            if (menuChoice != 1 && menuChoice != 2) {
                System.out.println("❌ Invalid option. Please select 1, 2, or 3.");
                continue;
            }

            System.out.print("\nEnter raw student name: ");
            String rawName = inputScanner.nextLine();

            Student_dataHiding currentStudent = null;

            if (menuChoice == 1) {
                currentStudent = new Student_dataHiding(rawName, inputScanner);
            } else if (menuChoice == 2) {
                System.out.print("Enter Research Thesis Topic: ");
                String thesis = inputScanner.nextLine();
                currentStudent = new GraduateStudent_inheritance(rawName, thesis, inputScanner);
            }

            // --- POLYMORPHIC REPORT COMPILATION ---
            System.out.println("\n=================================================");
            System.out.println("              GENERATING REPORT CARD             ");
            System.out.println("=================================================");
            System.out.println("Account File:    " + currentStudent.getName());
            System.out.println("Tracked Classes: " + currentStudent.getGrades().length);

            String extraDetails = "";
            if (currentStudent instanceof GraduateStudent_inheritance) {
                GraduateStudent_inheritance castedGrad = (GraduateStudent_inheritance) currentStudent;
                extraDetails = "Research Focus:  " + castedGrad.getThesisTitle() + "\n";
                System.out.print(extraDetails);
            }

            double averageGrade = currentStudent.calculateAverage();
            System.out.println("Final Grade:     " + averageGrade + "%");

            String honorStatus = "Standard Academic Standing";
            if (currentStudent.qualifiesForHonorRoll()) {
                honorStatus = "⭐ APPROVED FOR HONOR ROLL ⭐";
            }
            System.out.println("Honor Roll:      " + honorStatus);
            System.out.println("=================================================");

            // AUTOMATIC DATA BACKUP TO HARD DRIVE
            try (FileWriter fileWriter = new FileWriter("report_cards.txt", true)) {
                fileWriter.write("=== STUDENT RECORD ===\n");
                fileWriter.write("Name: " + currentStudent.getName() + "\n");
                fileWriter.write("Average: " + averageGrade + "%\n");
                if (!extraDetails.isEmpty()) {
                    fileWriter.write(extraDetails);
                }
                fileWriter.write("Status: " + honorStatus + "\n");
                fileWriter.write("-------------------------------------\n\n");

                System.out.println("💾 Backup Status: Report successfully written to 'report_cards.txt'.");

            } catch (IOException e) {
                System.out.println("❌ Local Storage Failure: Could not write record backup to disk drive.");
            }
        }

        inputScanner.close();
    }
}
```